Hito 10 — Requisitos Técnicos: API REST Integrada del Fixture 2030

Asignatura: Ingeniería de Datos II
Clase: 11 de 14
Tema: Acceso a bases NoSQL desde aplicaciones
Modelo: API REST integrada con adaptadores para las fuentes del Fixture 2030
Modalidad: Desarrollo grupal en la notebook del equipo
Duración prevista: 90 minutos de trabajo práctico
Repositorio: GitHub del equipo

---

1. Descripción general

La plataforma del Fixture Mundial 2030 utiliza distintos modelos NoSQL para resolver necesidades diferentes: información documental de equipos y jugadores, relaciones entre partidos y eventos, comentarios de gran volumen, sesiones y caché, estadísticas temporales y entidades complejas.

En este hito, el equipo deberá construir una **API REST integrada** que permita acceder a una parte representativa de esos módulos desde una aplicación. La API deberá ocultar los detalles internos de cada cliente o driver, ofrecer contratos HTTP claros y traducir las respuestas y errores de cada fuente a respuestas comprensibles para sus consumidores.

La API no debe convertir todas las bases en una única estructura ni aceptar consultas arbitrarias enviadas por el cliente. Cada endpoint deberá estar relacionado con un caso de uso del Fixture, una fuente o combinación de fuentes justificada, una operación concreta y un contrato documentado.

El diseño deberá conservar las diferencias entre modelos. Una consulta documental, un recorrido de grafo, una lectura por partición, una búsqueda de clave, una consulta temporal y una lectura de objetos no se ejecutan de la misma manera aunque todos se expongan mediante HTTP y JSON.

---

2. Objetivo del hito

Diseñar, implementar y documentar una API REST capaz de acceder a los módulos NoSQL del Fixture 2030 mediante adaptadores separados, con validación de entradas, contratos OpenAPI, manejo de errores, pruebas reproducibles y evidencia de conectividad y funcionamiento.

---

3. Requisitos funcionales

ID	Requisito funcional	Prioridad
RF1	Implementar una aplicación o servicio HTTP que exponga recursos relacionados con el Fixture Mundial 2030.	Alta
RF2	Identificar y documentar los casos de uso que la API debe resolver antes de definir las rutas y métodos HTTP.	Alta
RF3	Implementar endpoints para acceder a información documental de equipos o jugadores almacenada en MongoDB.	Alta
RF4	Implementar endpoints para recuperar partidos, eventos o relaciones almacenadas en Neo4j mediante una consulta Cypher parametrizada.	Alta
RF5	Implementar endpoints para consultar y, cuando corresponda al caso de uso, registrar comentarios en Cassandra respetando el patrón de partición definido en el hito correspondiente.	Alta

ID	Requisito funcional	Prioridad
RF6	Implementar un endpoint para consultar una sesión o dato temporal almacenado en Redis, contemplando la posibilidad de que la clave no exista o haya expirado.	Alta
RF7	Implementar un endpoint para recuperar estadísticas temporales desde InfluxDB, permitiendo acotar el intervalo o la agregación de acuerdo con el contrato definido por el equipo.	Alta
RF8	Implementar un endpoint para consultar una entidad compleja o una vista de objetos persistentes en IRIS, de acuerdo con el modelo desarrollado en la clase correspondiente.	Alta
RF9	Separar la lógica de entrada HTTP, reglas del caso de uso y acceso específico a cada base mediante módulos, servicios, repositorios o adaptadores claramente identificables.	Alta
RF10	Validar parámetros de ruta, parámetros de consulta y cuerpos JSON antes de ejecutar operaciones contra las bases.	Alta
RF11	Definir respuestas exitosas y respuestas de ausencia, validación, conflicto, dependencia no disponible y	Alta

ID	Requisito funcional	Prioridad
	error interno, según corresponda a cada operación.	
RF12	Implementar un formato de error consistente que permita identificar el tipo de problema sin exponer credenciales, consultas internas ni detalles sensibles.	Alta
RF13	Documentar la API mediante OpenAPI y permitir su exploración mediante Swagger UI o una herramienta equivalente.	Alta
RF14	Preparar una colección de Postman, ejemplos con cURL u otra herramienta reproducible para ejecutar las rutas principales y escenarios de error.	Alta
RF15	Implementar pruebas que demuestren al menos una operación exitosa, una entrada inválida, un recurso inexistente y una dependencia no disponible o temporalmente inaccesible.	Alta
RF16	Registrar la configuración necesaria para conectar la aplicación con las fuentes utilizadas, separando valores de ambiente y secretos del código versionado.	Alta
RF17	Registrar evidencia de que cada endpoint llega a la fuente o fuentes que le	Alta

ID	Requisito funcional	Prioridad
	corresponden y devuelve una respuesta compatible con el contrato documentado.	
RF18	Incorporar un identificador de request, correlación o mecanismo equivalente para relacionar la respuesta de la API con los logs del adaptador y de la aplicación.	Media
RF19	Explicar qué endpoints son de lectura, cuáles producen cambios y qué garantías de idempotencia, repetición o duplicación se aplican a las operaciones de escritura.	Media
RF20	Montar el código, la documentación, los contratos y las pruebas del hito en un repositorio GitHub del equipo.	Alta

---

4. Requisitos no funcionales

ID	Aspecto	Requisito verificable
RNF1	Separación de responsabilidades	Los controladores o rutas no deberán contener toda la lógica de conexión, consulta, transformación y manejo de errores en un único bloque.
RNF2	Contrato	Las rutas, métodos, parámetros, cuerpos, respuestas y errores implementados deberán coincidir con la documentación OpenAPI.

ID	Aspecto	Requisito verificable
RNF3	Validación	Las entradas inválidas deberán rechazarse antes de ejecutar una consulta o escritura innecesaria en la fuente de datos.
RNF4	Consistencia de respuestas	Las respuestas exitosas y de error deberán mantener una estructura coherente dentro de la API.
RNF5	Aislamiento de fuentes	Los detalles específicos de MongoDB, Neo4j, Cassandra, Redis, InfluxDB e IRIS deberán permanecer dentro del adaptador o módulo correspondiente.
RNF6	Configuración	Hosts, puertos, bases, keyspaces, namespaces, tokens y demás valores de conexión deberán configurarse mediante variables de entorno o archivos locales excluidos del repositorio.
RNF7	Seguridad	No se deberán publicar passwords, tokens, URI con credenciales, archivos `.env` sensibles ni secretos en código, logs, capturas o documentación.
RNF8	Timeouts	La API deberá definir o documentar límites de espera para las operaciones de red y dependencia cuando el cliente o framework lo permita.

ID	Aspecto	Requisito verificable
RNF9	Manejo de fallas	Una dependencia caída o no disponible no deberá dejar la request esperando indefinidamente ni devolver información técnica sensible al consumidor.
RNF10	Observabilidad básica	Los logs deberán permitir identificar ruta, método, request ID, dependencia, resultado y duración sin registrar secretos ni cuerpos sensibles innecesarios.
RNF11	Reproducibilidad	Un integrante ajeno al desarrollo deberá poder iniciar la API, configurar las dependencias disponibles y ejecutar las pruebas siguiendo el README.
RNF12	Paginación y límites	Las operaciones que puedan recuperar conjuntos grandes deberán utilizar límites, paginación, cursores, ventanas o una estrategia equivalente justificada.
RNF13	Diseño por acceso	Cada consulta deberá estar relacionada con un patrón de acceso concreto de la fuente utilizada. No se aceptarán endpoints que realicen recorridos o escaneos indiscriminados sin justificación.
RNF14	Idempotencia	Las operaciones que puedan repetirse por reintentos deberán documentar su comportamiento respecto de

ID	Aspecto	Requisito verificable
		duplicados, claves o cambios parciales.
RNF15	Legibilidad	Código, scripts, contratos, colección de pruebas y documentación deberán estar organizados y nombrados para poder localizar cada módulo.
RNF16	GitHub	La entrega deberá estar montada en un repositorio GitHub del equipo, con historial de commits, estructura navegable y acceso otorgado al docente según la modalidad indicada.
RNF17	Integridad	La API deberá transformar los datos sin perder identificadores, timestamps, estados o atributos necesarios para interpretar la respuesta.
RNF18	Rendimiento básico	Las mediciones de latencia o cantidad de requests deberán indicar método, entorno, volumen, configuración y limitaciones; no se aceptarán cifras sin evidencia.

---

5. Especificación técnica

5.1. Casos de uso y recursos

Antes de implementar, el equipo deberá documentar los casos de uso prioritarios. Para cada caso deberá indicar:

- quién consume la operación;
- qué información recibe o envía;
- qué fuente o fuentes participan;
- qué datos de entrada son necesarios;
- qué respuesta espera el consumidor;
- qué ocurre si el dato no existe;
- qué ocurre si la dependencia no está disponible;
- si la operación es de lectura, creación, actualización o invalidación;
- qué criterio define que la operación fue exitosa.

Las rutas deberán representar recursos o acciones de dominio comprensibles. No se aceptará un único endpoint genérico que reciba el nombre de una base, una colección y una consulta libre.

5.2. Arquitectura por adaptadores

La aplicación deberá mantener una separación reconocible entre:

Cliente HTTP

↓

Router o controlador

↓

Servicio / caso de uso

↓

Adaptador o repositorio específico

↓

Driver o cliente nativo

↓

Base NoSQL

El adaptador de cada producto deberá encapsular, como mínimo:

- creación o recepción de la conexión;

- selección de database, keyspace, namespace o contexto;
- ejecución de la operación;
- transformación de errores del cliente;
- conversión de la respuesta al modelo interno de la aplicación;
- cierre o administración de recursos según corresponda.

No se exige una única arquitectura de carpetas. Sí se exige que el código permita distinguir el contrato HTTP de la forma en que cada producto realiza la operación.

5.3. Fuentes del Fixture

La API deberá relacionarse con los módulos trabajados en las prácticas anteriores:

Fuente	**Módulo del Fixture**	**Ejemplo de capacidad a exponer**
MongoDB	Equipos y jugadores	Recuperación por código, filtro o página.
Neo4j	Partidos, participantes y eventos	Consulta de relaciones y eventos conectados.
Cassandra	Comentarios	Lectura paginada y, si corresponde, creación de comentarios.
Redis	Sesiones y caché	Lectura, invalidación o estado temporal.
InfluxDB	Estadísticas temporales	Ventana de tiempo y agregación.
IRIS	Entidades complejas	Detalle o transición validada del dominio.

La tabla orienta la cobertura de productos; no define las rutas finales ni la implementación que cada equipo deberá elegir.

5.4. Contrato HTTP y OpenAPI

La especificación OpenAPI deberá documentar como mínimo:

- título y versión del servicio;
- servidor o URL local;

- tags por módulo o recurso;
- rutas y métodos;
- parámetros de ruta y query;
- cuerpos de request cuando corresponda;
- schemas de respuesta;
- códigos HTTP posibles;
- esquema de error;
- ejemplos representativos;
- mecanismo de autenticación si el equipo lo implementa.

La documentación deberá ser coherente con el comportamiento observado. No se aceptará una especificación que describa endpoints que no existen o respuestas que la aplicación nunca produce.

5.5. Acceso a MongoDB

El adaptador documental deberá:

- utilizar el driver o cliente apropiado;
- seleccionar la database configurada;
- ejecutar consultas relacionadas con los patrones definidos en el hito documental;
- limitar o paginar resultados que puedan crecer;
- proyectar los campos que la respuesta realmente necesita cuando corresponda;
- traducir documento inexistente a una respuesta de ausencia coherente;
- evitar entregar sin criterio documentos internos completos.

5.6. Acceso a Neo4j

El adaptador de grafos deberá:

- utilizar una sesión y contexto de base definidos;
- ejecutar Cypher parametrizado;
- limitar recorridos y resultados según el caso de uso;
- distinguir partido inexistente de consulta válida sin eventos;
- transformar nodos y relaciones a una respuesta JSON comprensible;
- evitar aceptar Cypher arbitrario desde parámetros enviados por el consumidor.

5.7. Acceso a Cassandra

El adaptador tabular deberá:

- seleccionar el keyspace correcto;
- respetar la tabla y la clave de partición diseñadas en el hito de comentarios;
- utilizar límites, paginación o cursores cuando el conjunto pueda crecer;

- documentar el comportamiento de una escritura repetida;
- evitar consultas que ignoren el patrón de partición sin una justificación técnica;
- traducir errores de disponibilidad o timeout sin ocultar el problema en un `200` vacío.

5.8. Acceso a Redis

El adaptador clave/valor deberá:

- utilizar el host, puerto y credenciales configurados;
- contemplar TTL y expiración de las sesiones;
- diferenciar clave inexistente de Redis no disponible;
- documentar el comportamiento de invalidación;
- evitar exponer datos internos de sesión que no necesita el consumidor;
- definir el código HTTP correspondiente ante una sesión ausente o expirada.

5.9. Acceso a InfluxDB

El adaptador temporal deberá:

- seleccionar la database y autorización local configuradas;
- acotar las consultas por intervalo temporal y dimensiones relevantes;
- validar fechas y orden de los límites recibidos;
- diferenciar datos sin puntos de una dependencia no disponible;
- documentar la agregación o resolución temporal solicitada;
- evitar devolver un volumen ilimitado de puntos en una única respuesta.

5.10. Acceso a IRIS

El adaptador de objetos deberá:

- utilizar el namespace y credenciales configurados;
- consultar la entidad o clase correspondiente al módulo construido;
- preservar identificadores y relaciones necesarias para la respuesta;
- validar las transiciones o cambios de estado antes de persistirlos;
- diferenciar entidad inexistente, transición inválida y dependencia caída;
- evitar publicar detalles internos del modelo de objetos que no formen parte del contrato.

5.11. Validación y errores

La API deberá distinguir, cuando corresponda:

Situación	Respuesta esperada a definir
Parámetro ausente o con formato incorrecto	Error de validación, sin consultar innecesariamente la base.
Recurso inexistente	Respuesta de ausencia coherente con el contrato.
Sesión expirada	Respuesta que indique que no existe una sesión vigente.
Conflicto de negocio	Respuesta que explique que la operación no está permitida por el estado actual.
Dependencia no disponible	Error de dependencia, timeout controlado o respuesta degradada documentada.
Error inesperado	Error interno seguro, con detalle técnico únicamente en logs.

El código HTTP concreto deberá estar documentado y ser consistente con la implementación. No se deberá devolver `200` para ocultar una falla de conexión.

5.12. Pruebas de la API

La entrega deberá incluir pruebas reproducibles para:

- una request válida por módulo cubierto;
- un parámetro inválido;
- un recurso inexistente;
- una respuesta vacía válida, si corresponde;
- una sesión ausente o expirada;
- una dependencia detenida o inaccesible;
- una operación de escritura repetida, si el endpoint la implementa;
- límites, paginación o ventanas temporales;
- documentación y respuesta real compatibles.

Las pruebas podrán ejecutarse mediante framework de tests, Postman, Newman, cURL, Swagger UI u otra herramienta adecuada. El método elegido deberá quedar explicado en el README.

5.13. Conectividad y entorno

La guía de ejecución deberá explicar:

- cómo iniciar los servicios necesarios;
- cómo configurar las variables de entorno;
- cómo verificar que cada dependencia está disponible;
- cómo iniciar la API;
- cómo abrir Swagger UI, si está disponible;
- cómo importar y ejecutar la colección Postman;
- cómo ejecutar las pruebas sin exponer secretos;
- cómo detener la aplicación y los servicios sin borrar datos indebidamente.

La prueba desde la notebook y la prueba desde un contenedor pueden requerir hosts y puertos diferentes. La documentación deberá indicar desde qué entorno se ejecuta cada componente.

---

6. Restricciones

Restricción	**Aplicación**
Caso	Todos los recursos, ejemplos y datos de la API deberán referirse al Fixture Mundial 2030.
Cobertura	La API deberá cubrir los módulos NoSQL que el equipo haya desarrollado y pueda ejecutar, documentando cualquier fuente que no esté disponible.
Diseño	No se aceptará un endpoint que exponga consultas libres, colecciones arbitrarias, Cypher enviado por el cliente o comandos NoSQL sin control.
Arquitectura	La aplicación deberá separar rutas, lógica de caso de uso y adaptadores o módulos de acceso a datos.
Contrato	Las rutas y respuestas deberán documentarse en OpenAPI y coincidir con el comportamiento real.

Restricción	Aplicación
Datos	Se utilizarán datos de laboratorio, sintéticos o generados por el equipo. No se deberán incorporar datos sensibles.
Seguridad	No se deberán versionar credenciales, tokens, URI con passwords, archivos `.env` sensibles ni secretos.
Integración	No se exige que una única request consulte todas las bases. Cada integración deberá justificarse por el caso de uso.
Rendimiento	No se deberán presentar cifras de rendimiento sin describir método, volumen, configuración y entorno.
Errores	No se deberán ocultar fallas de las dependencias devolviendo respuestas exitosas o listas vacías sin justificación.
Repositorio	Código, contratos, pruebas, documentación y colección deberán estar en GitHub, con `.gitignore` y estructura navegable.
Entrega	No se solicita presentación, defensa oral ni video. La evaluación se realizará sobre el repositorio, la API, la documentación y las pruebas.

---

7. Notas importantes

Una API no elimina las diferencias entre bases NoSQL. Las encapsula para ofrecer una interfaz de aplicación, pero cada adaptador debe respetar el modelo, las consultas y las garantías del producto que utiliza.

HTTP 200 no significa que la dependencia funcionó correctamente. Una respuesta vacía puede ser válida, pero no debe utilizarse para esconder un timeout, una credencial inválida o una base caída.

La validación debe ocurrir antes de consultar. Un parámetro incorrecto no debería producir una consulta costosa o ambigua contra una fuente de datos.

Las credenciales pertenecen al entorno, no al repositorio. El código deberá poder ejecutarse con variables locales sin publicar secretos.

La API debe controlar la superficie de acceso. No se deberá permitir que el consumidor decida arbitrariamente qué colección, tabla, keyspace, nodo, relación o comando ejecutar.

La idempotencia depende de la operación. Repetir un GET no debería producir efectos; repetir una creación o comentario puede requerir una clave de idempotencia o una decisión explícita sobre duplicados.

La disponibilidad de una base no garantiza disponibilidad de la API. La aplicación debe definir timeouts, traducción de errores y comportamiento ante dependencias caídas.

Una documentación que no coincide con la aplicación no es un contrato. OpenAPI, Swagger UI, Postman y las pruebas deberán mantenerse alineados.

---

8. Estructura esperada del análisis

La documentación del hito deberá permitir comprender el diseño y las decisiones de integración. Como mínimo, deberá incluir:

Apartado	**Contenido esperado**
Casos de uso	Consumidores, operaciones, datos de entrada, respuesta y fuente involucrada.
Arquitectura	Capas de la aplicación, responsabilidades y relación entre routers, servicios y adaptadores.
Mapa de endpoints	Rutas, métodos, fuente, operación y caso de uso asociado.
Contratos	OpenAPI, schemas, parámetros, ejemplos y códigos de estado.

Apartado	Contenido esperado
Adaptadores	Forma de conexión, contexto, consulta, transformación y errores por producto.
Validación	Reglas para rutas, query parameters, cuerpos JSON, fechas, límites y cursores.
Errores	Formato común, diferencias entre ausencia, validación, conflicto y dependencia caída.
Conectividad	Hosts, puertos, variables, procedimiento de prueba y diferencias entre notebook y Docker.
Idempotencia	Operaciones repetibles, duplicados, invalidación y escritura segura.
Pruebas	Casos exitosos, inválidos, ausentes, errores de dependencia y límites.
Seguridad	Manejo de secretos, logs y archivos excluidos del repositorio.
Rendimiento básico	Método de medición, volumen, latencia observada y limitaciones del entorno.
Coherencia con el TPO	Relación con los módulos y decisiones de los hitos anteriores.

---

9. Entregables

El material deberá estar montado en el repositorio GitHub del equipo, en una carpeta claramente identificable para el Hito 10. La siguiente organización es una referencia mínima; los nombres pueden variar si el contenido resulta localizable y reproducible.

fixture2030-hito-10-api-integrada/

├── README.md

├── app/

```
│   ├── routes/             # rutas o controladores HTTP
│   ├── services/           # casos de uso
│   ├── adapters/           # clientes de MongoDB, Neo4j, Cassandra, Redis, InfluxDB e IRIS
│   ├── schemas/            # validación y modelos de request/response
│   └── config/             # configuración sin secretos
├── openapi/
│   └── openapi.yaml
├── postman/
│   └── fixture2030_collection.json
├── tests/
│   ├── unit/
│   ├── integration/
│   └── README.md
├── scripts/
│   ├── iniciar_api.*
│   ├── verificar_conectividad.*
│   └── ejecutar_pruebas.*
├── docs/
│   ├── casos_de_uso.md
│   ├── arquitectura_y_adaptadores.md
│   ├── contratos_y_errores.md
│   ├── conectividad_y_configuracion.md
```

```
│   ├── pruebas_y_evidencia.md
│   └── evidencia/
└── .gitignore
```

El repositorio deberá incluir:

- código fuente de la API;
- README con requisitos, instalación, configuración y orden de ejecución;
- contrato OpenAPI actualizado;
- colección Postman, ejemplos cURL o mecanismo equivalente;
- adaptadores y consultas parametrizadas;
- validación de entradas;
- pruebas exitosas y de error;
- documentación de conectividad;
- evidencias sin secretos;
- `.gitignore` adecuado;
- historial de commits que permita identificar la evolución del trabajo.

No se deberán subir al repositorio:

- passwords;
- tokens;
- URI con credenciales;
- archivos `.env` con secretos;
- dumps de datos innecesarios;
- logs con información sensible;
- claves privadas;
- respuestas copiadas sin documentar cómo fueron obtenidas.

---

10. Criterios de calidad

Se considerará que el hito está técnicamente bien resuelto cuando:

- la API representa casos de uso claros del Fixture 2030;
- las rutas y métodos son coherentes con los recursos expuestos;
- la lógica HTTP está separada del acceso específico a las fuentes;
- cada adaptador respeta el modelo del producto correspondiente;
- los parámetros y cuerpos se validan antes de consultar;

- la documentación OpenAPI coincide con la implementación;
- Swagger UI, Postman o cURL permiten reproducir los escenarios;
- se diferencian respuestas exitosas, ausencia, validación, conflicto y dependencia caída;
- las respuestas no exponen documentos, errores ni secretos innecesarios;
- existen timeouts o límites documentados;
- las consultas están acotadas y relacionadas con patrones de acceso;
- se comprueba la conectividad de las fuentes utilizadas;
- se documenta el comportamiento de las operaciones de escritura repetidas;
- el repositorio es reproducible y no contiene credenciales;
- las métricas declaradas tienen método y evidencia.

---

11. Rúbrica de evaluación docente

Dimensión	**Evidencia esperada**
Casos de uso	La API responde a necesidades concretas del Fixture y no a consultas genéricas.
Arquitectura	Existe separación clara entre HTTP, lógica de aplicación y adaptadores.
Integración NoSQL	Los clientes, contextos y consultas respetan las particularidades de cada producto.
Contrato OpenAPI	Las rutas, parámetros, schemas, respuestas y errores están documentados y son coherentes.
Validación y errores	Se controlan entradas, ausencias, conflictos, timeouts y dependencias no disponibles.
Pruebas	Hay escenarios exitosos, inválidos, inexistentes y de falla de dependencia reproducibles.
Conectividad	La aplicación puede configurarse y probarse contra las fuentes disponibles.
Seguridad	No se publican secretos y los logs son técnicamente útiles sin exponer información sensible.

Dimensión	Evidencia esperada
Reproducibilidad	README, scripts, colección y estructura permiten que otra persona repita el trabajo.
Evolución y documentación	El repositorio GitHub muestra orden, trazabilidad y explicación de decisiones.

La evaluación priorizará la comprensión de la integración, la calidad del contrato y la capacidad de explicar las decisiones técnicas por sobre la cantidad de endpoints.

---

12. Preguntas guía

Antes de cerrar la entrega, el equipo debería poder responder:

1. ¿Qué problema del Fixture resuelve cada endpoint?
2. ¿Qué fuente atiende cada operación y por qué?
3. ¿Qué parte del código conoce el driver específico de cada base?
4. ¿Qué ocurre si el recurso no existe?
5. ¿Qué ocurre si la base está caída?
6. ¿Cómo se distingue una respuesta vacía válida de un error de dependencia?
7. ¿Qué parámetros deben validarse antes de consultar?
8. ¿Qué límites o paginación se aplican a las respuestas grandes?
9. ¿Qué endpoint modifica datos y cómo se comporta si se repite?
10. ¿Cómo se protege una operación de escritura frente a reintentos o duplicados?
11. ¿Qué diferencia hay entre consultar Neo4j mediante una relación y consultar Cassandra mediante una partición?
12. ¿Qué diferencia hay entre una clave Redis inexistente y Redis no disponible?
13. ¿Qué intervalo temporal acepta el endpoint de InfluxDB?
14. ¿Qué namespace y entidad utiliza el adaptador IRIS?
15. ¿Cómo se configura cada host y puerto sin modificar el código?
16. ¿Qué información se registra en los logs y qué información se omite?
17. ¿Cómo se verifica que OpenAPI describe el comportamiento real?
18. ¿Qué prueba ejecutaría el equipo si una dependencia responde lentamente?
19. ¿Qué parte de la API podría cachearse y qué riesgo de frescura tendría?
20. ¿Qué limitaciones del laboratorio impiden afirmar que la API está lista para millones de usuarios?

---

13. Próximos pasos

Después de completar el Hito 10, el equipo deberá conservar una API que pueda utilizarse como punto de entrada para:

- evaluar la conectividad de las fuentes;
- ejecutar pruebas reproducibles desde Postman o Swagger UI;
- cargar y consultar volúmenes mayores;
- medir latencia y errores;
- integrar métricas y observabilidad;
- definir escenarios de carga y optimización en los hitos siguientes.

La API debe considerarse una capa de integración controlada, no una excusa para ocultar decisiones de modelado ni para exponer directamente la implementación interna de cada base.